Experiment 6 – JWT Generation After Signin

Question: Implement JWT-based authentication for the E-Commerce application and generate a JWT token successfully after valid user signin.

Requirements:

1. Accept registered email and password through the signin API.
2. Validate the user credentials.
3. Generate a JWT token after successful authentication.
4. Return the JWT token in the signin response.
5. Test the signin API using Postman and verify the generated token.

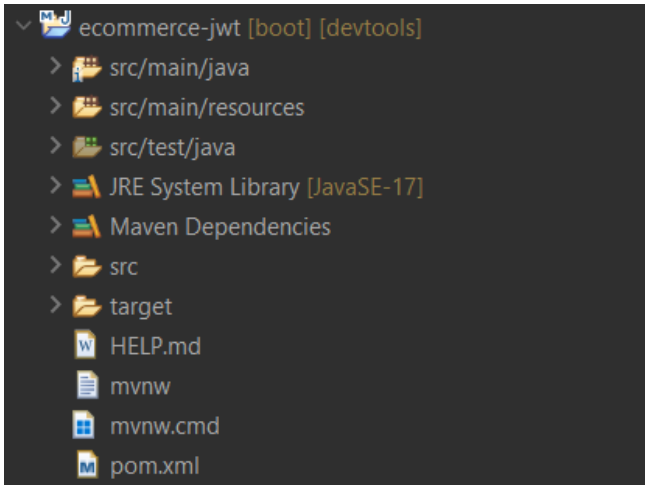
Main Concepts: User authentication, JWT, signin, token generation.

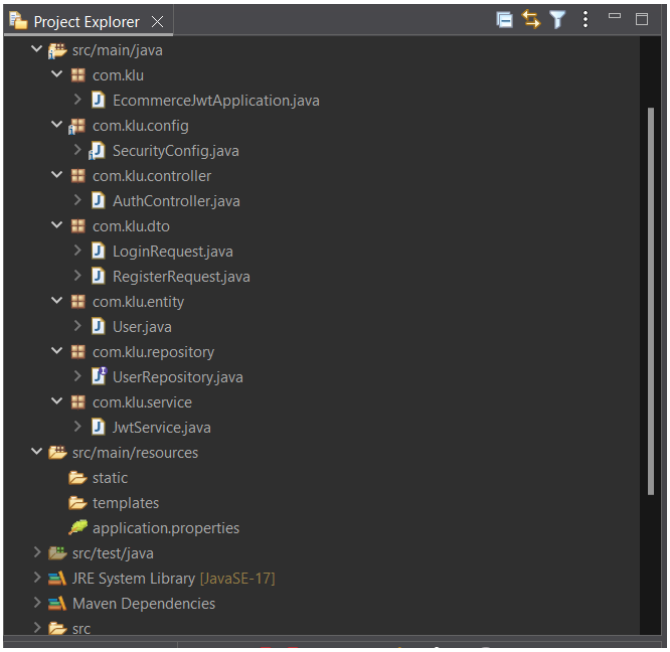
->We use Backend **SpringBoot and PGAdmin (PostGreSQL)**for this Experiment

SpringBoot:

Create a new Spring Stater Project By name -> "ecommerce-jwt"

Folder Structure:





Application.properties:

spring.application.name=ecommerce-jwt

spring.datasource.url=jdbc:postgresql://localhost:5432/ecommerce_jwt

spring.datasource.username=postgres

```
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
server.port=8080
```

EcommerceJwtApplication:

```
package com.klu;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class EcommerceJwtApplication {
  public static void main(String[] args) {
    SpringApplication.run(EcommerceJwtApplication.class, args);
  }
}
```

SecurityConfig:

```
package com.klu.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
@Configuration
public class SecurityConfig {
  @Bean
  public SecurityFilterChain securityFilterChain(
```

```
        HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
          .requestMatchers(
            "/api/auth/register",
            "/api/auth/signin"
          ).permitAll()
          .anyRequest().authenticated()
        );
      return http.build();
    }
    @Bean
    public BCryptPasswordEncoder passwordEncoder() {
      return new BCryptPasswordEncoder();
    }
}
```

AuthController:

```
package com.klu.controller;
import com.klu.dto.LoginRequest;
import com.klu.dto.RegisterRequest;
import com.klu.entity.User;
import com.klu.repository.UserRepository;
import com.klu.service.JwtService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.web.bind.annotation.*;
```

```
import java.util.Map;
@RestController
@RequestMapping("/api/auth")
@CrossOrigin(origins = "*")
public class AuthController {
  private final UserRepository userRepository;
  private final JwtService jwtService;
  private final BCryptPasswordEncoder passwordEncoder;
  public AuthController(
      UserRepository userRepository,
      JwtService jwtService,
      BCryptPasswordEncoder passwordEncoder) {
    this.userRepository = userRepository;
    this.jwtService = jwtService;
    this.passwordEncoder = passwordEncoder;
  }
  @PostMapping("/register")
  public ResponseEntity<?> register(
      @RequestBody RegisterRequest request) {
    if (userRepository
        .findByEmail(request.getEmail())
        .isPresent()) {
      return ResponseEntity.badRequest()
          .body(Map.of(
              "message",
              "Email already registered"
          ));
```

```
    }
    User user = new User();
    user.setName(request.getName());
    user.setEmail(request.getEmail());
    user.setPassword(
        passwordEncoder.encode(
            request.getPassword()
        )
    );
    userRepository.save(user);
    return ResponseEntity.ok(
        Map.of(
            "message",
            "User registered successfully"
        )
    );
}
@PostMapping("/signin")
public ResponseEntity<?> signin(
    @RequestBody LoginRequest request) {
    User user = userRepository
        .findByEmail(request.getEmail())
        .orElse(null);
    if (user == null) {
      return ResponseEntity.status(401)
          .body(Map.of(
              "message",
              "Invalid email or password"
```

```
            ));
        }
        boolean validPassword =
                passwordEncoder.matches(
                        request.getPassword(),
                        user.getPassword()
                );
        if (!validPassword) {
          return ResponseEntity.status(401)
                  .body(Map.of(
                          "message",
                          "Invalid email or password"
                  ));
        }
        String token =
                jwtService.generateToken(
                        user.getEmail()
                );
        return ResponseEntity.ok(
                Map.of(
                        "message", "Login successful",
                        "email", user.getEmail(),
                        "token", token
                )
        );
    }
}
```

LoginRequest:

```
package com.klu.dto;
public class LoginRequest {
  private String email;
  private String password;
  public String getEmail() {
    return email;
  }
  public void setEmail(String email) {
    this.email = email;
  }
  public String getPassword() {
    return password;
  }
  public void setPassword(String password) {
    this.password = password;
  }
}
```

RegisterRequest:

```
package com.klu.dto;
public class RegisterRequest {
  private String name;
  private String email;
  private String password;
  public String getName() {
    return name;
  }
```

```
    public void setName(String name) {
        this.name = name;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}
```

User.java:

```
package com.klu.entity;
import jakarta.persistence.*;
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
```

```
    @Column(unique = true, nullable = false)
    private String email;
    @Column(nullable = false)
    private String password;
    public User() {
    }
    public Long getId() {
       return id;
    }
    public void setId(Long id) {
       this.id = id;
    }
    public String getName() {
       return name;
    }
    public void setName(String name) {
       this.name = name;
    }
    public String getEmail() {
       return email;
    }
    public void setEmail(String email) {
       this.email = email;
    }
    public String getPassword() {
       return password;
    }
   public void setPassword(String password) {
```

```
        this.password = password;
    }
}
```

UserRepository:

```
package com.klu.repository;
import com.klu.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
public interface UserRepository
        extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}
```

Imp-> JwtService:

```
package com.klu.service;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.stereotype.Service;
import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
@Service
public class JwtService {
    private final String SECRET =
            "mySuperSecretKeyForJwtAuthentication123456789";
```

```
    private final SecretKey key =
            Keys.hmacShaKeyFor(
                SECRET.getBytes(StandardCharsets.UTF_8)
            );
    public String generateToken(String email) {
        return Jwts.builder()
                .subject(email)
                .issuedAt(new Date())
                .expiration(
                    new Date(
                        System.currentTimeMillis()
                            + 86400000
                    )
                )
                .signWith(key)
                .compact();
    }
}
```

After Updating all the files "Run" the Backend :8080 and Open Postman Application

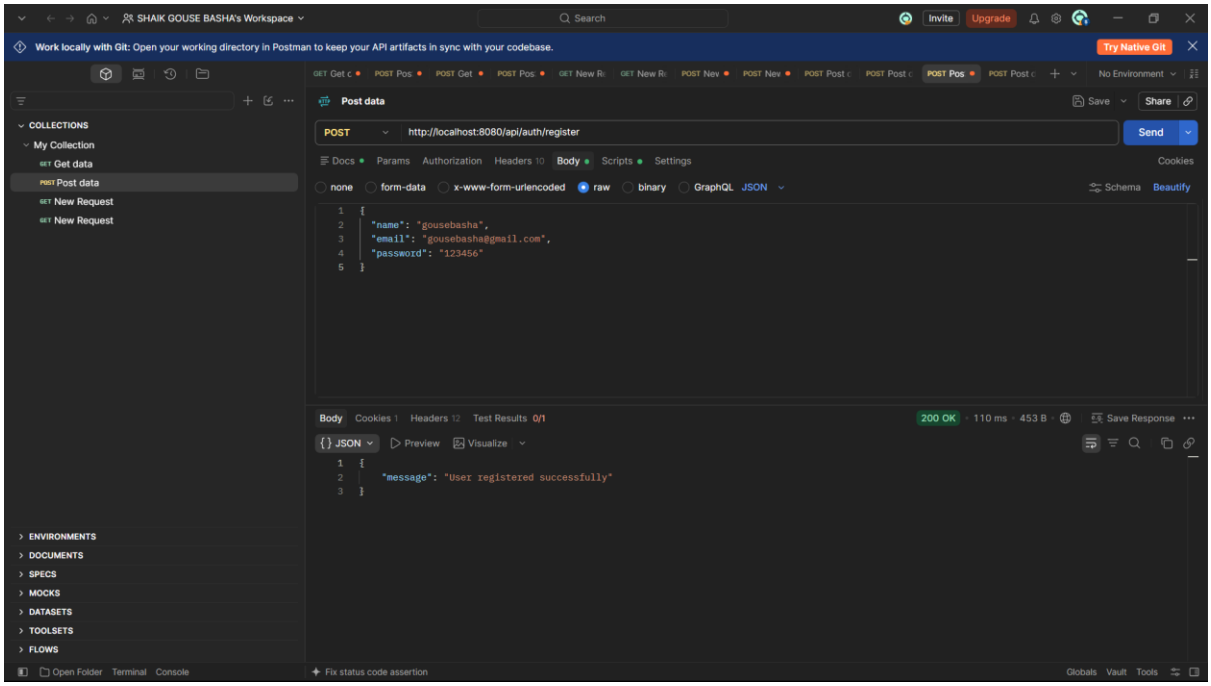
Register the Admin:

POST http://localhost:8080/api/auth/register

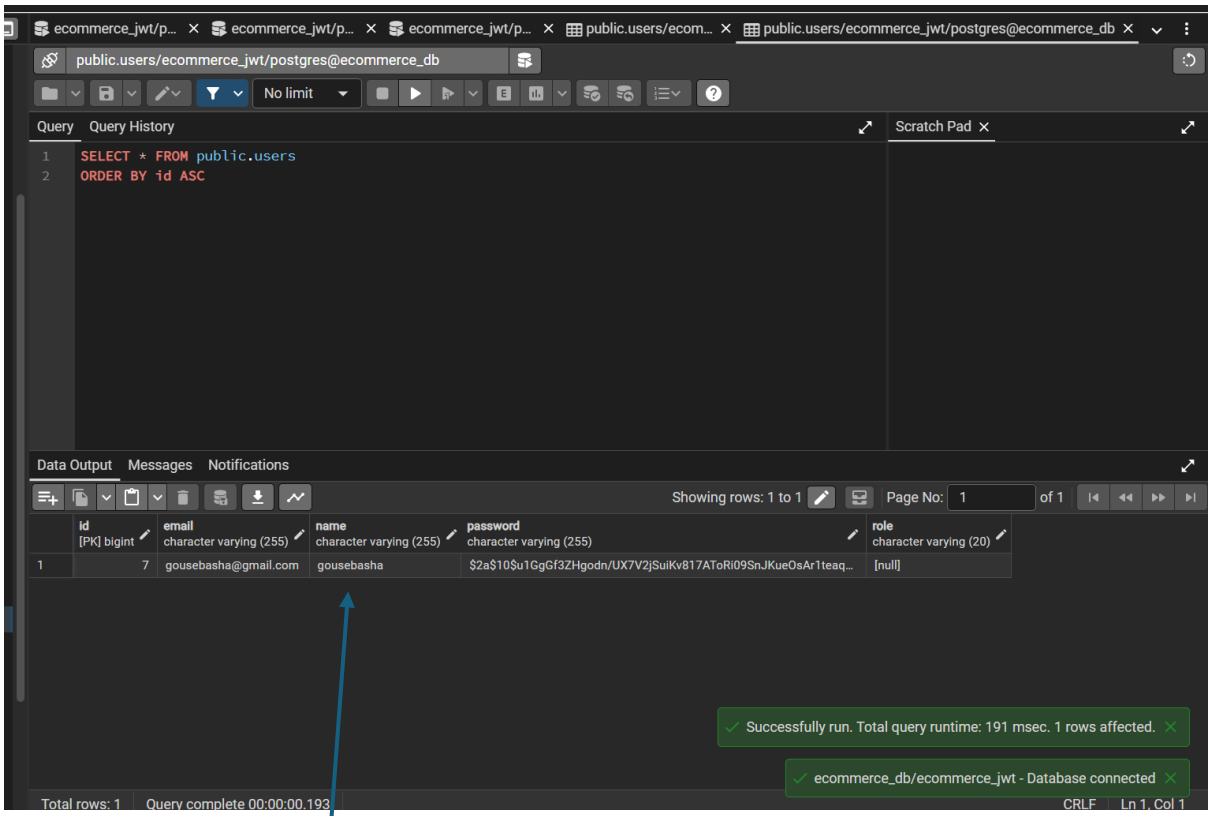
Enter the Details in Body->Raw:

```
{
  "name": "gousebasha",
  "email": "gousebasha@gmail.com",
  "password": "123456"
}
```

“Click Send”



Now Check the Database:



Refresh the “users” and Chek the Details are Stored

Now Signin :

POST http://localhost:8080/api/auth/signin

Enter the email and password in Body->Raw:

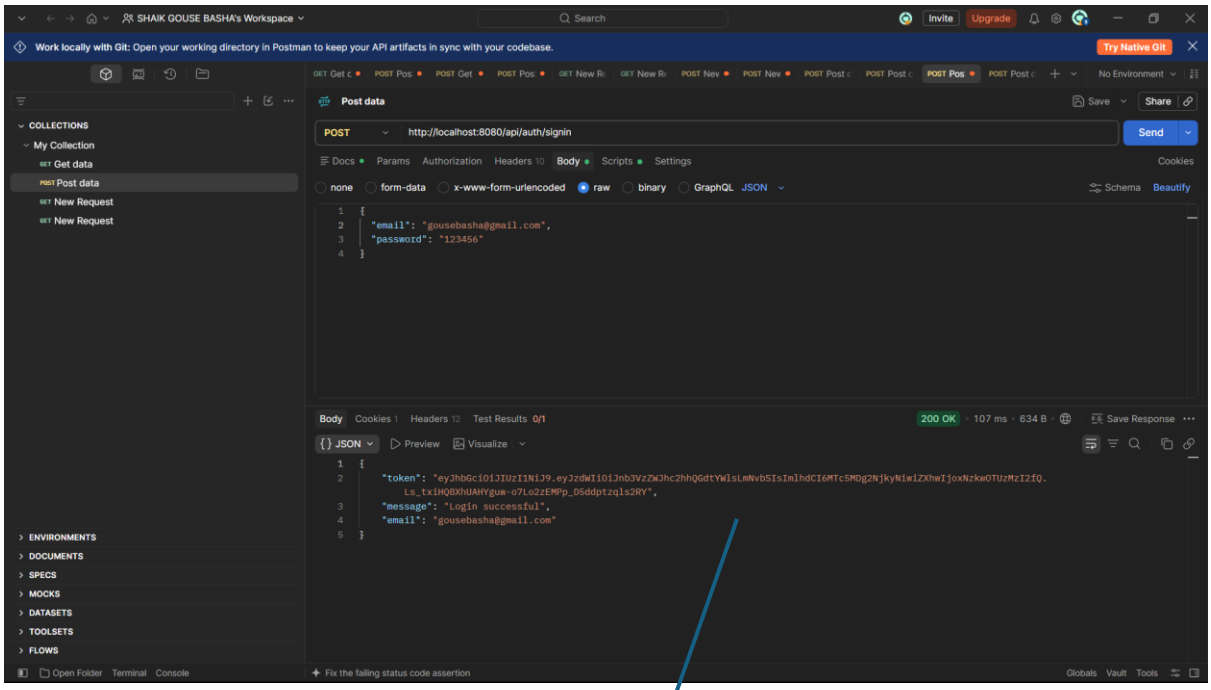
{

"email": "gousebasha@gmail.com",

"password": "123456"

}

CLICK “send”



Token

Copy the generated token:

{

"token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJnb3VzZWJhc2hhQGdtYWlsLmNvbSIsImlhdCI6MTc5MDg2NjkyNiwiZXhwIjoxNzkwOTUzMzI2fQ.Ls_txiHQ8XhUAHYguw-o7Lo2zEMPp_D5ddptzqls2RY",

"message": "Login successful",

"email": "gousebasha@gmail.com"

}

PAST IT IN Authorization-> Bearer Token -> PAST

